

Smart Contract Security Audit Report

Table Of Contents

1 Executive Summary

2 Audit Methodology

3 Project Overview

3.1 Project Introduction

3.2 Vulnerability Information

4 Code Overview

4.1 Contracts Description

4.2 Visibility Description

4.3 Vulnerability Summary

5 Audit Result

6 Statement

1 Executive Summary

On 2026.09.22, the SlowMist security team received the HashKeyChain team's security audit application for htp-smartcontract, developed the audit plan according to the agreement of both parties and the characteristics of the project, and finally issued the security audit report.

The SlowMist security team adopts the strategy of "white box lead, black, grey box assists" to conduct a complete security test on the project in the way closest to the real attack.

The test method information:

Test method	Description
Black box testing	Conduct security tests from an attacker's perspective externally.
Grey box testing	Conduct security testing on code modules through the scripting tool, observing the internal running status, mining weaknesses.
White box testing	Based on the open source code, non-open source code, to detect whether there are vulnerabilities in programs such as nodes, SDK, etc.

The vulnerability severity level information:

Level	Description
Critical	Critical severity vulnerabilities will have a significant impact on the security of the DeFi project, and it is strongly recommended to fix the critical vulnerabilities.
High	High severity vulnerabilities will affect the normal operation of the DeFi project. It is strongly recommended to fix high-risk vulnerabilities.
Medium	Medium severity vulnerability will affect the operation of the DeFi project. It is recommended to fix medium-risk vulnerabilities.
Low	Low severity vulnerabilities may affect the operation of the DeFi project in certain scenarios. It is suggested that the project team should evaluate and consider whether these vulnerabilities need to be fixed.
Weakness	There are safety risks theoretically, but it is extremely difficult to reproduce in engineering.

Level	Description
Suggestion	There are better practices for coding or architecture.

2 Audit Methodology

The security audit process of SlowMist security team for smart contract includes two steps:

- Smart contract codes are scanned/tested for commonly known and more specific vulnerabilities using automated analysis tools.
- Manual audit of the codes for security issues. The contracts are manually analyzed to look for any potential problems.

Following is the list of commonly known vulnerabilities that was considered during the audit of the smart contract:

Serial Number	Audit Class	Audit Subclass
1	Overflow Audit	-
2	Reentrancy Attack Audit	-
3	Replay Attack Audit	-
4	Flashloan Attack Audit	-
5	Race Conditions Audit	Reordering Attack Audit
6	Permission Vulnerability Audit	Access Control Audit
		Excessive Authority Audit
7	Security Design Audit	External Module Safe Use Audit
		Compiler Version Security Audit

Serial Number	Audit Class	Audit Subclass
7	Security Design Audit	Hard-coded Address Security Audit
		Fallback Function Safe Use Audit
		Show Coding Security Audit
		Function Return Value Security Audit
		External Call Function Security Audit
		Block data Dependence Security Audit
		tx.origin Authentication Security Audit
8	Denial of Service Audit	-
9	Gas Optimization Audit	-
10	Design Logic Audit	-
11	Variable Coverage Vulnerability Audit	-
12	"False Top-up" Vulnerability Audit	-
13	Scoping and Declarations Audit	-
14	Malicious Event Log Audit	-
15	Arithmetic Accuracy Deviation Audit	-
16	Uninitialized Storage Pointer Audit	-

3 Project Overview

3.1 Project Introduction

The project is a Delivery-versus-Payment (DvP) settlement system for the HashKey Trading Platform. The system comprises a core settlement engine (DvPSettlement) with signed-payload authorization, pluggable asset adapters (ERC1404 restricted tokens, USDY, unrestricted ERC20), principal calculators for bond/fund/stock instruments (day-count and interest accrual math), a suite of UUPS-upgradeable configuration registries (fees, market rules, eligibility, participants, settlement tokens, adapters, principal calculators, asset versions), settlement state validation and strict failure mapping, and supporting cryptographic/hashing, fee math and parameter validation libraries.

3.2 Vulnerability Information

The following is the status of the vulnerabilities found in this audit:

NO	Title	Category	Level	Status
N1	Missing Fee Version Readiness Check During Fee Epoch Activation	Design Logic Audit	Suggestion	Fixed
N2	Deprecated Fee Versions Remain Executable but Cannot Be Individually Paused	Design Logic Audit	Medium	Fixed
N3	Zero-Address Admin Bypasses Last Admin Protection	Design Logic Audit	Low	Fixed
N4	Proxy Pattern Detection Cannot Fully Establish a Contract's Non-Proxy Status	Design Logic Audit	Information	Fixed
N5	Settlement Token Decimal Validation Is Inconsistent with the Range Supported by Market Rules	Design Logic Audit	Suggestion	Fixed
N6	Inconsistent Gas Budgets Between	Design Logic Audit	Suggestion	Fixed

NO	Title	Category	Level	Status
	Calculator Validation and Settlement			
N7	Backdated Snapshot Time Allows a Limited Bypass of the Market Rule Quote Window	Design Logic Audit	Information	Acknowledged
N8	Closing a Market Rule Clears Its Pause Flag Without Emitting the Corresponding Event	Design Logic Audit	Suggestion	Fixed
N9	Settlement Token Adapter Failure Reports the Asset Adapter ID	Design Logic Audit	Low	Fixed
N10	Missing Maker Signature Binding Allows Unilateral Trade ID Replacement by the Requester	Design Logic Audit	Medium	Fixed
N11	Market Rule Violations Are Misreported as External Call Failures During Strict Settlement	Design Logic Audit	Low	Fixed
N12	Participant Type Validation Order Makes the Dedicated Error Branch Unreachable	Others	Suggestion	Fixed
N13	Missing Nonzero Evidence Hash Validation When Creating or Updating Participants	Others	Suggestion	Fixed
N14	ERC-1967 Slot Constant Scanning May Reject Legitimate Direct Contracts	Others	Information	Acknowledged

NO	Title	Category	Level	Status
N15	Risk of Excessive Privilege	Authority Control Vulnerability Audit	Medium	Acknowledged

4 Code Overview

4.1 Contracts Description

https://github.com/HSKChain/htp-smartcontract

Initial Audit Version: 4318d0443f8e7683f3bb72d8d0e2b00508cd33d1

Final Audit Version: 0455328716234beb173583bfd5aab35ee11a9bd2

Audit Scope:

```
contracts/src/
├── DvPSettlement.sol
├── adapters/
│   ├── ERC1404Adapter.sol
│   ├── UnrestrictedERC20Adapter.sol
│   └── UsdyRestrictedAdapter.sol
├── calculators/
│   ├── BondPrincipalCalculator.sol
│   ├── FundPrincipalCalculator.sol
│   └── StockPrincipalCalculator.sol
├── interfaces/
│   ├── IAdapterRegistry.sol
│   ├── IAssetAdapter.sol
│   ├── IAssetVersionRegistry.sol
│   ├── IDvPCurrentStateValidator.sol
│   ├── IDvPCurrentStateValidatorFactory.sol
│   ├── IDvPGovernanceView.sol
│   ├── IDvPSettlement.sol
│   ├── IDvPSignedPayloadValidator.sol
│   ├── IEligibilityRegistry.sol
│   ├── IFeeRegistry.sol
```

```
│   ├── IMarketRuleRegistry.sol
│   ├── InterfaceVersions.sol
│   ├── IParticipantRegistry.sol
│   ├── IPrincipalCalculator.sol
│   ├── IPrincipalCalculatorRegistry.sol
│   └── ISettlementTokenRegistry.sol
├── libraries/
│   ├── AdapterInputValidation.sol
│   ├── AdapterReasonCodes.sol
│   ├── BoundedStaticCall.sol
│   ├── CalendarDate.sol
│   ├── DeploymentVersion.sol
│   ├── FeeMath.sol
│   ├── FeeScope.sol
│   ├── HTPHashing.sol
│   ├── PrincipalMath.sol
│   ├── ProtocolSecurityLimits.sol
│   ├── RegistryImplementationValidation.sol
│   ├── RestrictionProviderLib.sol
│   ├── Roles.sol
│   ├── SettlementParamsValidation.sol
│   └── TokenMetadataValidation.sol
├── registry/
│   ├── AdapterRegistry.sol
│   ├── AssetVersionRegistry.sol
│   ├── EligibilityRegistry.sol
│   ├── FeeRegistry.sol
│   ├── LinearConfigRegistryBase.sol
│   ├── MarketRuleRegistry.sol
│   ├── ParticipantRegistry.sol
│   ├── PrincipalCalculatorRegistry.sol
│   ├── SettlementTokenRegistry.sol
│   └── UUPSRegistryBase.sol
├── types/
│   ├── HTPErrors.sol
│   ├── HTPEvents.sol
│   └── HTPTypes.sol
└── validation/
    ├── ConfigurationReferenceIntegrity.sol
    ├── DvPCurrentStateValidator.sol
    ├── DvPCurrentStateValidatorFactory.sol
    ├── DvPSignedPayloadValidator.sol
    └── StrictSettlementFailureMapper.sol
```

The main network address of the contract is as follows:

The code was not deployed to the mainnet.

4.2 Visibility Description

The SlowMist Security team analyzed the visibility of major contracts during the audit, the result as follows:

ERC1404Adapter			
Function Name	Visibility	Mutability	Modifiers
<Constructor>	Public	Can Modify State	-
interfaceVersion	External	-	-
capabilities	External	-	-
capabilityHash	External	-	-
providerHash	External	-	-
preCheck	External	-	-
_hasPermissiveFallback	Private	-	-
_capabilities	Private	-	-

UnrestrictedERC20Adapter			
Function Name	Visibility	Mutability	Modifiers
<Constructor>	Public	Can Modify State	-
interfaceVersion	External	-	-
capabilities	External	-	-
capabilityHash	External	-	-

UnrestrictedERC20Adapter			
providerHash	External	-	-
preCheck	External	-	-
_capabilities	Private	-	-

UsdyRestrictedAdapter			
Function Name	Visibility	Mutability	Modifiers
<Constructor>	Public	Can Modify State	-
interfaceVersion	External	-	-
capabilities	External	-	-
capabilityHash	External	-	-
providerHash	External	-	-
preCheck	External	-	-
_providersMatch	Private	-	-
_checkAccount	Private	-	-
_readBool	Private	-	-
_readAddress	Private	-	-
_capabilities	Private	-	-

BondPrincipalCalculator			
Function Name	Visibility	Mutability	Modifiers
interfaceVersion	External	-	-

BondPrincipalCalculator			
assetClass	External	-	-
calculate	External	-	-
_decodePriceData	Private	-	-

FundPrincipalCalculator			
Function Name	Visibility	Mutability	Modifiers
interfaceVersion	External	-	-
assetClass	External	-	-
calculate	External	-	-
_decodePriceData	Private	-	-

StockPrincipalCalculator			
Function Name	Visibility	Mutability	Modifiers
interfaceVersion	External	-	-
assetClass	External	-	-
calculate	External	-	-
_decodePriceData	Private	-	-

DvPSettlement			
Function Name	Visibility	Mutability	Modifiers
<Constructor>	Public	Can Modify State	-

DvPSettlement			
_grantRole	Internal	Can Modify State	-
_revokeRole	Internal	Can Modify State	-
domainSeparatorV4	Public	-	-
preCheck	External	-	-
getNonceStatus	Public	-	-
cancelNonce	External	Can Modify State	nonReentrant
cancelNonceBySig	External	Can Modify State	nonReentrant
pauseGlobal	External	Can Modify State	onlyRole(Roles.PAUSER_ROLE) nonReentrant
unpauseGlobal	External	Can Modify State	onlyRole(Roles.GOVERNANCE_ROLE) nonReentrant
retire	External	Can Modify State	onlyRole(Roles.GOVERNANCE_ROLE) nonReentrant
settle	External	Can Modify State	-
_settle	Private	Can Modify State	nonReentrant
_validateSettlementCurrentState	Private	-	-
_usedReplayHash	Private	-	-
_validateReplayAvailability	Private	-	-
_writeExecutionLocks	Private	Can Modify State	-
_requireUnusedNonce	Private	-	-

DvPSettlement			
_tryPermits	Private	Can Modify State	-
_tryPermit	Private	Can Modify State	-
_snapshotBalancesAndRequireFunds	Private	-	-
_snapshotBalances	Private	-	-
_readTokenWord	Private	-	-
_executeTransfers	Private	Can Modify State	-
_safeTransferFrom	Private	Can Modify State	-
_validateBalanceDeltas	Private	-	-
_requireDecrease	Private	-	-
_requireIncrease	Private	-	-
_requireUnchanged	Private	-	-
_emitSettlementEvents	Private	Can Modify State	-
_requireRegistry	Private	-	-
_validateAdapters	Private	-	-
_revertAdapterFailure	Private	-	-
_callAdapter	Private	-	-
_requireReasonAndEvidence	Private	-	-
_cancelNonce	Private	Can Modify State	-

DvPSettlement			
_requireRoleHolder	Private	-	-

AdapterRegistry			
Function Name	Visibility	Mutability	Modifiers
initialize	External	Can Modify State	initializer
registerAdapter	External	Can Modify State	onlyRole(Roles.CONFIGURER_ROLE)
activateAdapter	External	Can Modify State	onlyRole(Roles.GOVERNANCE_ROLE)
pauseAdapter	External	Can Modify State	onlyRole(Roles.PAUSER_ROLE)
unpauseAdapter	External	Can Modify State	onlyRole(Roles.GOVERNANCE_ROLE)
deprecateAdapter	External	Can Modify State	onlyRole(Roles.GOVERNANCE_ROLE)
getAdapter	External	-	-
getCurrentAdapter	External	-	-
isAdapterActive	External	-	-
_recomputeLinearConfigHash	Internal	-	-
_validateConfigIdentity	Private	-	-
_validateAdapter	Private	-	-
_validateAdapterVersion	Private	-	-
_validateAdapterCapabilities	Private	-	-
_validateAdapterCapabilityHash	Private	-	-

AdapterRegistry			
_validateAdapterProviderHash	Private	-	-
_requireAdapterCallResult	Private	-	-
_validateCapabilities	Private	-	-
_emitAdapterRegistered	Private	Can Modify State	-
_changeStatus	Private	Can Modify State	-
_adapterRecord	Private	-	-
_requireAdapterExists	Private	-	-
_getAdapterRegistryStorage	Private	-	-

AssetVersionRegistry			
Function Name	Visibility	Mutability	Modifiers
initialize	External	Can Modify State	initializer
registerAssetVersion	External	Can Modify State	onlyRole(Roles.CONFIGURER_ROLE)
activateAssetVersion	External	Can Modify State	onlyRole(Roles.GOVERNANCE_ROLE)
pauseAssetVersion	External	Can Modify State	onlyRole(Roles.PAUSER_ROLE)
unpauseAssetVersion	External	Can Modify State	onlyRole(Roles.GOVERNANCE_ROLE)
deprecateAssetVersion	External	Can Modify State	onlyRole(Roles.GOVERNANCE_ROLE)
getAssetVersion	External	-	-
getCurrentAssetVersion	External	-	-

AssetVersionRegistry			
getEconomicAssetIdHash	External	-	-
_recomputeLinearConfigHash	Internal	-	-
_validateConfigIdentity	Private	-	-
_validateToken	Private	-	-
_validateReferences	Private	-	-
_validateDvp	Private	-	-
_readCanonicalBoolResult	Private	-	-
_emitAssetVersionRegistered	Private	Can Modify State	-
_assetVersionRecord	Private	-	-
_tokenSubject	Private	-	-
_requireAssetVersionExists	Private	-	-
_getAssetVersionRegistryStorage	Private	-	-

EligibilityRegistry			
Function Name	Visibility	Mutability	Modifiers
initialize	External	Can Modify State	initializer
setEligibility	External	Can Modify State	onlyRole(Roles.CONFIGURER_ROLE)
pauseEligibility	External	Can Modify State	onlyRole(Roles.PAUSER_ROLE)
unpauseEligibility	External	Can Modify State	onlyRole(Roles.GOVERNANCE_ROLE)

EligibilityRegistry			
revokeEligibility	External	Can Modify State	onlyRole(Roles.GOVERNANCE_ROLE)
getEligibility	External	-	-
isEligible	External	-	-
_eligibilityKey	Private	-	-
_requireEligibilityVersion	Private	-	-
_emitEligibilityUpdated	Private	Can Modify State	-
_getEligibilityRegistryStorage	Private	-	-

FeeRegistry			
Function Name	Visibility	Mutability	Modifiers
initialize	External	Can Modify State	initializer
createFeePolicyEpoch	External	Can Modify State	onlyRole(Roles.GOVERNANCE_ROLE)
sealFeePolicyEpoch	External	Can Modify State	onlyRole(Roles.GOVERNANCE_ROLE)
activateFeePolicyEpoch	External	Can Modify State	onlyRole(Roles.GOVERNANCE_ROLE)
moveFeePolicyEpochToLegacyServing	External	Can Modify State	onlyRole(Roles.GOVERNANCE_ROLE)
closeFeePolicyEpoch	External	Can Modify State	onlyRole(Roles.GOVERNANCE_ROLE)
registerFeeVersion	External	Can Modify State	onlyRole(Roles.CONFIGURER_ROLE)
activateFeeVersion	External	Can Modify State	onlyRole(Roles.GOVERNANCE_ROLE)

FeeRegistry			
pauseFeeVersion	External	Can Modify State	onlyRole(Roles.PAUSER_ROLE)
unpauseFeeVersion	External	Can Modify State	onlyRole(Roles.GOVERNANCE_ROLE)
deprecateFeeVersion	External	Can Modify State	onlyRole(Roles.GOVERNANCE_ROLE)
getFeePolicyEpoch	External	-	-
getFeeVersion	External	-	-
resolveApplicableFee	External	-	-
isFeeVersionExecutable	External	-	-
_findApplicableFee	Private	-	-
_validateFeeConfigIdentity	Private	-	-
_validateFeeScope	Private	-	-
_validateFeeTerms	Private	-	-
_validateSettlementToken	Private	-	-
_isSettlementTokenValid	Private	-	-
_requireFeeVersionInMutableEpoch	Private	-	-
_requireStoredFeeConfigHash	Private	-	-
_emitFeeVersionStatusChanged	Private	Can Modify State	-
_emitFeePolicyEpochCreated	Private	Can Modify State	-
_emitFeePolicyEpochStatusChanged	Private	Can Modify State	-

FeeRegistry			
_emitFeeVersionRegistered	Private	Can Modify State	-
_requireEpochConfigHashes	Private	-	-
_transitionExpiredEpoch	Private	Can Modify State	-
_hashEpochCalldata	Private	-	-
_hashEpochMemory	Private	-	-
_hashFeeConfigCalldata	Private	-	-
_hashFeeConstraintCalldata	Private	-	-
_getFeeRegistryStorage	Private	-	-

LinearConfigRegistryBase			
Function Name	Visibility	Mutability	Modifiers
_registerLinearConfig	Internal	Can Modify State	-
_activateLinearConfig	Internal	Can Modify State	-
_unpauseLinearConfig	Internal	Can Modify State	-
_transitionLinearConfig	Internal	Can Modify State	-
_linearConfigMetadata	Internal	-	-
_currentLinearConfigId	Internal	-	-
_recomputeLinearConfigHash	Internal	-	-
_requireConfigHashes	Private	-	-
_requireExisting	Private	-	-

LinearConfigRegistryBase			
_getLinearConfigRegistryStorage	Private	-	-

MarketRuleRegistry			
Function Name	Visibility	Mutability	Modifiers
initialize	External	Can Modify State	initializer
registerMarketRule	External	Can Modify State	onlyRole(Roles.CONFIGURER_ROLE)
sealMarketRule	External	Can Modify State	onlyRole(Roles.GOVERNANCE_ROLE)
activateMarketRule	External	Can Modify State	onlyRole(Roles.GOVERNANCE_ROLE)
pauseMarketRule	External	Can Modify State	onlyRole(Roles.PAUSER_ROLE)
unpauseMarketRule	External	Can Modify State	onlyRole(Roles.GOVERNANCE_ROLE)
closeMarketRule	External	Can Modify State	onlyRole(Roles.GOVERNANCE_ROLE)
getMarketRule	External	-	-
getCurrentMarketRule	External	-	-
isMarketRuleServiceable	External	-	-
_validateIdentity	Private	-	-
_validateReferencesAndRules	Private	-	-
_requireConfigHash	Private	-	-
_requireRecord	Private	-	-
_requireServiceableStatus	Private	-	-

MarketRuleRegistry			
_getMarketRuleRegistryStorage	Private	-	-

ParticipantRegistry			
Function Name	Visibility	Mutability	Modifiers
initialize	External	Can Modify State	initializer
setParticipant	External	Can Modify State	onlyRole(Roles.CONFIGURER_ROLE)
getParticipant	External	-	-
isParticipantActive	External	-	-
pauseParticipant	External	Can Modify State	onlyRole(Roles.PAUSER_ROLE)
unpauseParticipant	External	Can Modify State	onlyRole(Roles.GOVERNANCE_ROLE)
revokeParticipant	External	Can Modify State	onlyRole(Roles.GOVERNANCE_ROLE)
_getParticipantRegistryStorage	Private	-	-
_validateParticipantInput	Private	-	-
_requireParticipantVersion	Private	-	-
_emitParticipantUpdated	Private	Can Modify State	-

PrincipalCalculatorRegistry			
Function Name	Visibility	Mutability	Modifiers
initialize	External	Can Modify State	initializer

PrincipalCalculatorRegistry			
registerPrincipalCalculator	External	Can Modify State	onlyRole(Roles.CONFIGURER_ROLE)
activatePrincipalCalculator	External	Can Modify State	onlyRole(Roles.GOVERNANCE_ROLE)
pausePrincipalCalculator	External	Can Modify State	onlyRole(Roles.PAUSER_ROLE)
unpausePrincipalCalculator	External	Can Modify State	onlyRole(Roles.GOVERNANCE_ROLE)
deprecatePrincipalCalculator	External	Can Modify State	onlyRole(Roles.GOVERNANCE_ROLE)
getPrincipalCalculator	External	-	-
getCurrentPrincipalCalculator	External	-	-
isPrincipalCalculatorConfigActive	External	-	-
_recomputeLinearConfigHash	Internal	-	-
_validateConfigIdentity	Private	-	-
_validateCalculator	Private	-	-
_requireCalculatorCallResult	Private	-	-
_probePriceData	Private	-	-
_changeStatus	Private	Can Modify State	-
_calculatorRecord	Private	-	-
_assetClassSubject	Private	-	-
_isValidAssetClass	Private	-	-
_requireCalculatorExists	Private	-	-

PrincipalCalculatorRegistry			
_getPrincipalCalculatorRegistryStorage	Private	-	-

SettlementTokenRegistry			
Function Name	Visibility	Mutability	Modifiers
initialize	External	Can Modify State	initializer
registerSettlementToken	External	Can Modify State	onlyRole(Roles.CONFIGURER_ROLE)
activateSettlementToken	External	Can Modify State	onlyRole(Roles.GOVERNANCE_ROLE)
pauseSettlementToken	External	Can Modify State	onlyRole(Roles.PAUSER_ROLE)
unpauseSettlementToken	External	Can Modify State	onlyRole(Roles.GOVERNANCE_ROLE)
deprecateSettlementToken	External	Can Modify State	onlyRole(Roles.GOVERNANCE_ROLE)
getSettlementToken	External	-	-
getCurrentSettlementToken	External	-	-
isSettlementTokenConfigActive	External	-	-
isSettlementTokenConfigRegisterable	External	-	-
_recomputeLinearConfigHash	Internal	-	-
_validateConfigIdentity	Private	-	-
_validateToken	Private	-	-
_isTokenValid	Private	-	-
_validateAdapterReference	Private	-	-

SettlementTokenRegistry			
_emitSettlementTokenRegistered	Private	Can Modify State	-
_changeStatus	Private	Can Modify State	-
_settlementTokenRecord	Private	-	-
_tokenSubject	Private	-	-
_requireSettlementTokenExists	Private	-	-
_getSettlementTokenRegistryStorage	Private	-	-

UUPSRegistryBase			
Function Name	Visibility	Mutability	Modifiers
<Constructor>	Public	Can Modify State	-
_initializeRegistryBase	Internal	Can Modify State	onlyInitializing
_grantRole	Internal	Can Modify State	-
_revokeRole	Internal	Can Modify State	-
_authorizeUpgrade	Internal	Can Modify State	onlyRole(Roles.UPGRADER_ROLE)
registerDependentDvp	External	Can Modify State	onlyRole(Roles.GOVERNANCE_ROLE)
getDependentDvpCount	External	-	-
exemptRetiredDependentDvp	External	Can Modify State	onlyRole(Roles.GOVERNANCE_ROLE)
getDependentDvp	External	-	-

UUPSRegistryBase			
_isDependentDvpRegistered	Internal	-	-
_requireReasonAndEvidence	Internal	-	-
_getRegistryBaseStorage	Private	-	-
_requireRoleHolder	Private	-	-
_strictBoolStaticCall	Private	-	-

ConfigurationReferenceIntegrity			
Function Name	Visibility	Mutability	Modifiers
loadAssetVersion	External	-	-
loadSettlementToken	External	-	-
loadAdapters	External	-	-
loadCalculator	External	-	-
loadFee	External	-	-
validateMarketRule	External	-	-
_assetVersionMatches	Private	-	-
_feeRecordsServiceable	Private	-	-
_adapterActive	Private	-	-
_errorSelector	Private	-	-

DvPCurrentStateValidator			
Function Name	Visibility	Mutability	Modifiers

DvPCurrentStateValidator			
<Constructor>	Public	Can Modify State	-
preCheck	External	-	-
validateForSettlement	External	-	-
_validate	Private	-	-
_validateForSettlementStrict	Private	-	-
_requireSettlementPermitFields	Private	-	-
_validateInputs	Private	-	-
_validateNonces	Private	-	-
_loadAndValidateConfiguration	Private	-	-
_validateTokenMetadata	Private	-	-
_validatePrincipalAndFees	Private	-	-
_validateEligibilityAndAllowance	Private	-	-
_validateAllowance	Private	-	-
_revertFailure	Private	-	-
_addressesAreDisjoint	Private	-	-
_quantityFailureReason	Private	-	-
_feeAmountsMatch	Private	-	-
_readWord	Private	-	-
_unusedPermitIsCanonical	Private	-	-
_pass	Private	-	-

DvPCurrentStateValidator			
_block	Private	-	-
_externalUnknown	Private	-	-

DvPCurrentStateValidatorFactory			
Function Name	Visibility	Mutability	Modifiers
<Constructor>	Public	Can Modify State	-
deployValidator	External	Can Modify State	-

DvPSignedPayloadValidator			
Function Name	Visibility	Mutability	Modifiers
<Constructor>	Public	Can Modify State	-
preCheck	External	-	onlyDvp
validateForSettlement	External	-	onlyDvp
validateNonceCancellation	External	-	onlyDvp
_check	Private	-	-
_checkSignature	Private	-	-
_revertFailure	Private	-	-
_hashFailure	Private	-	-
_invalidField	Private	-	-

StrictSettlementFailureMapper			
Function Name	Visibility	Mutability	Modifiers

StrictSettlementFailureMapper			
revertFailure	External	-	-
requireNonces	External	-	-
validateParticipants	External	-	-
requireParticipants	External	-	-
validateEligibility	External	-	-
requireEligibility	External	-	-
permitFailure	External	-	-
requirePermit	External	-	-
revertTokenMetadataFailure	External	-	-
_participantValidation	Private	-	-
_eligibilityValidation	Private	-	-
_revertEligibilityFailure	Private	-	-
_permitFailure	Private	-	-
_participantActive	Private	-	-

4.3 Vulnerability Summary

[N1] [Suggestion] Missing Fee Version Readiness Check During Fee Epoch Activation

Category: Design Logic Audit

Content

In the `FeeRegistry` contract, `activateFeePolicyEpoch()` allows governance to activate a fee epoch before any fee version has been activated, while `activateFeeVersion()` can only be called when the epoch is in the

`DRAFT` state. An incorrect operation order may therefore produce an epoch that cannot be used for settlement and within which fee versions can no longer be activated. Restoring the configuration requires creating another epoch, increasing operational costs.

- contracts/src/registry/FeeRegistry.sol#L187-L219, L275-L282

```
    function activateFeePolicyEpoch(uint64 feePolicyEpoch, bytes32 expectedConfigHash)
        external
        onlyRole(Roles.GOVERNANCE_ROLE)
    {
        FeeRegistryStorage storage $ = _getFeeRegistryStorage();
        FeePolicyEpochRecord storage record = $.epochs[feePolicyEpoch];
        if (!record.exists) revert HTPErrors.FeePolicyEpochNotFound(feePolicyEpoch);
        if (!record.isSealed) revert
HTPErrors.FeePolicyEpochNotSealed(feePolicyEpoch);
        if (record.status != FeePolicyEpochStatus.DRAFT) {
            revert HTPErrors.UnauthorizedStateTransition(
                bytes32(uint256(feePolicyEpoch)), uint8(record.status),
uint8(FeePolicyEpochStatus.ACTIVE)
            );
        }
        if (block.timestamp < record.config.quoteValidFrom || block.timestamp >
record.config.quoteValidUntil) {
            revert HTPErrors.FeePolicyEpochOutsideQuoteWindow(feePolicyEpoch,
uint64(block.timestamp));
        }
        _requireEpochConfigHashes(record.config, expectedConfigHash);

        uint64 oldActiveEpoch = $.currentActiveEpoch;
        if (oldActiveEpoch != 0) {
            FeePolicyEpochRecord storage oldActive = $.epochs[oldActiveEpoch];
            oldActive.status = FeePolicyEpochStatus.LEGACY_SERVING;
            _emitFeePolicyEpochStatusChanged(
                oldActiveEpoch, FeePolicyEpochStatus.ACTIVE,
FeePolicyEpochStatus.LEGACY_SERVING, bytes32(0)
            );
```

```
        }

        record.status = FeePolicyEpochStatus.ACTIVE;
        $.currentActiveEpoch = feePolicyEpoch;
        _emitFeePolicyEpochStatusChanged(
            feePolicyEpoch, FeePolicyEpochStatus.DRAFT, FeePolicyEpochStatus.ACTIVE,
bytes32(0)
        );
    }
```

```
        FeeVersionRecord storage record =
_requireFeeVersionInMutableEpoch(feeVersionId);
        FeeRegistryStorage storage $ = _getFeeRegistryStorage();
        FeePolicyEpochRecord storage epoch = $.epochs[record.config.feePolicyEpoch];
        if (epoch.status != FeePolicyEpochStatus.DRAFT) {
            revert HTPErrors.UnauthorizedStateTransition(
                feeVersionId, uint8(epoch.status), uint8(FeePolicyEpochStatus.DRAFT)
            );
        }
```

Solution

Before activating a fee epoch, verify that the required fee versions are activated and usable, or add a corresponding readiness check to the governance release process to prevent premature epoch activation.

Status

Fixed

[N2] [Medium] Deprecated Fee Versions Remain Executable but Cannot Be Individually Paused

Category: Design Logic Audit

Content

In the `FeeRegistry` contract, `deprecateFeeVersion()` changes a fee version from `ACTIVE` to `DEPRECATED` without changing its serving-version pointer or configured validity period. In `ConfigurationReferenceIntegrity`, `_feeRecordsServiceable()` explicitly accepts `DEPRECATED` versions,

while `pauseFeeVersion()` only permits pausing `ACTIVE` versions. The existing state-transition functions also provide no transition out of `DEPRECATED`. Consequently, when the resolver still selects the version, its epoch is `ACTIVE` or `LEGACY_SERVING`, and the validity and other settlement checks pass, both existing and newly created trades may continue using it, but an account with pause authority can no longer pause that version individually.

Once an epoch is sealed, no additional fee versions can be registered. Since `activateFeeVersion()` can change the serving version only while the epoch is `DRAFT`, an epoch already in service cannot replace the affected version by registering a version for a higher-priority scope or activating a previously registered alternative. If an incorrect fee configuration is discovered after deprecation or an emergency stop becomes necessary, operators must rely on other blocking measures, such as pausing the DVP or related dependencies, which may also affect trades using unaffected fee versions.

- contracts/src/registry/FeeRegistry.sol#L246-L248, L275-L282, L302-L317, L345-L359

```
        FeePolicyEpochRecord storage epoch = $.epochs[config.feePolicyEpoch];
        if (!epoch.exists) revert
HTPErrors.FeePolicyEpochNotFound(config.feePolicyEpoch);
        if (epoch.isSealed) revert
HTPErrors.FeePolicyEpochSealed(config.feePolicyEpoch);

        FeeVersionRecord storage record =
_requireFeeVersionInMutableEpoch(feeVersionId);
        FeeRegistryStorage storage $ = _getFeeRegistryStorage();
        FeePolicyEpochRecord storage epoch = $.epochs[record.config.feePolicyEpoch];
        if (epoch.status != FeePolicyEpochStatus.DRAFT) {
            revert HTPErrors.UnauthorizedStateTransition(
                feeVersionId, uint8(epoch.status), uint8(FeePolicyEpochStatus.DRAFT)
            );
        }

    function pauseFeeVersion(bytes32 feeVersionId, ReasonCode reason, bytes32
evidenceHash)
        external
```

```
        onlyRole(Roles.PAUSER_ROLE)
    {
        FeeVersionRecord storage record =
_requireFeeVersionInMutableEpoch(feeVersionId);
        _requireReasonAndEvidence(reason, evidenceHash);
        if (record.status != FeeVersionStatus.ACTIVE) {
            revert HTPErrors.UnauthorizedStateTransition(
                feeVersionId, uint8(record.status), uint8(FeeVersionStatus.PAUSED)
            );
        }
        record.status = FeeVersionStatus.PAUSED;
        _emitFeeVersionStatusChanged(
            record.config, FeeVersionStatus.ACTIVE, FeeVersionStatus.PAUSED, reason,
evidenceHash
        );
    }

    function deprecateFeeVersion(bytes32 feeVersionId, ReasonCode reason, bytes32
evidenceHash)
        external
        onlyRole(Roles.GOVERNANCE_ROLE)
    {
        FeeVersionRecord storage record =
_requireFeeVersionInMutableEpoch(feeVersionId);
        _requireReasonAndEvidence(reason, evidenceHash);
        FeeVersionStatus oldStatus = record.status;
        if (oldStatus != FeeVersionStatus.ACTIVE) {
            revert HTPErrors.UnauthorizedStateTransition(
                feeVersionId, uint8(oldStatus), uint8(FeeVersionStatus.DEPRECATED)
            );
        }
        record.status = FeeVersionStatus.DEPRECATED;
        _emitFeeVersionStatusChanged(record.config, oldStatus,
FeeVersionStatus.DEPRECATED, reason, evidenceHash);
    }
```

- contracts/src/validation/ConfigurationReferenceIntegrity.sol#L228-L250

```
    function _feeRecordsServiceable(
        FeeReferences calldata expected,
        FeePolicyEpochRecord memory epoch,
        FeeVersionRecord memory fee
    ) private view returns (bool) {
        bool epochServiceable = epoch.exists && epoch.isSealed
            && (epoch.status == FeePolicyEpochStatus.ACTIVE || epoch.status ==
FeePolicyEpochStatus.LEGACY_SERVING);
        bool feeServiceable =
            fee.exists && (fee.status == FeeVersionStatus.ACTIVE || fee.status ==
FeeVersionStatus.DEPRECATED);
        return epochServiceable && feeServiceable && epoch.config.feePolicyEpoch ==
expected.policyEpoch
            && fee.config.feeVersionId == expected.versionId &&
fee.config.feePolicyEpoch == expected.policyEpoch
            && fee.config.matchedScopeHash == expected.matchedScopeHash
            && fee.config.feeConstraintHash == expected.constraintHash
            && fee.config.settlementToken == expected.settlementToken
            && fee.config.settlementTokenConfigId == expected.settlementTokenConfigId
            && fee.config.settlementTokenConfigHash ==
expected.settlementTokenConfigHash
            && fee.config.collector == expected.collector && fee.config.roundingMode
== RoundingMode.DOWN
            && expected.roundingMode == RoundingMode.DOWN && fee.config.roundingMode
== expected.roundingMode
            && fee.config.validFrom <= expected.createdAt && expected.createdAt <=
block.timestamp
            && block.timestamp <= expected.tradeDeadline && expected.tradeDeadline <=
expected.quoteExpiresAt
            && expected.quoteExpiresAt <= fee.config.validUntil
            && expected.tradeDeadline <= epoch.config.settlementValidUntil;
    }
```

Solution

Confirm whether changing a fee version to `DEPRECATED` while allowing it to remain usable for settlement within its validity period and subject to all other checks is consistent with the intended design. Based on that confirmation,

define the settlement permissions and disabling rules for deprecated fee versions.

Status

Fixed

[N3] [Low] Zero-Address Admin Bypasses Last Admin Protection

Category: Design Logic Audit

Content

1.In `UUPSRegistryBase`, role holders are checked for nonzero addresses only during initialization. Subsequent calls to `_grantRole()` directly invoke the parent implementation and update `defaultAdminCount`. The project's `AccessControlUpgradeable` checks only the caller's role administration permission in `grantRole()`, while its internal `_grantRole()` does not reject the zero address. Consequently, the sole admin can call `grantRole(DEFAULT_ADMIN_ROLE, address(0))` to increase the admin count from `1` to `2`, then remove its own admin role through `renounceRole()` or `revokeRole()`, bypassing the last admin protection in `_revokeRole()`. The count remains `1`, but only the zero address holds the admin role, leaving no operational admin and disabling normal role administration.

2. `DvPSettlement` has the same issue: nonzero role addresses are checked only in the constructor, while subsequent calls to `_grantRole()` invoke `AccessControl._grantRole()`, which accepts the zero address, and include it in `_defaultAdminCount`. The sole admin can likewise grant `DEFAULT_ADMIN_ROLE` to the zero address and then remove its own role through either `renounceRole()` or `revokeRole()`; both paths have been reproduced. Because `DvPSettlement` is non-upgradeable, losing the last operational admin leaves no way to restore role administration through the contract's existing interfaces. This requires action by an existing admin and is a misconfiguration protection flaw, not privilege escalation by an external attacker.

- contracts/src/registry/UUPSRegistryBase.sol#L55-L68

```
    function _grantRole(bytes32 role, address account) internal override returns (bool
granted) {
```

```
        granted = super._grantRole(role, account);
        if (granted && role == Roles.DEFAULT_ADMIN_ROLE)
++_getRegistryBaseStorage().defaultAdminCount;
    }

    /// @dev Prevents removal of the last default admin.
    function _revokeRole(bytes32 role, address account) internal override returns
(bool revoked) {
        RegistryBaseStorage storage $ = _getRegistryBaseStorage();
        if (role == Roles.DEFAULT_ADMIN_ROLE && $.defaultAdminCount == 1 &&
hasRole(role, account)) {
            revert HTPErrors.LastDefaultAdminCannotBeRemoved(account);
        }
        revoked = super._revokeRole(role, account);
        if (revoked && role == Roles.DEFAULT_ADMIN_ROLE) --$.defaultAdminCount;
    }
```

- contracts/lib/openzeppelin-contracts-upgradeable/contracts/access/AccessControlUpgradeable.sol#L203-L212

```
    function _grantRole(bytes32 role, address account) internal virtual returns (bool)
{
        AccessControlStorage storage $ = _getAccessControlStorage();
        if (!hasRole(role, account)) {
            $._roles[role].hasRole[account] = true;
            emit RoleGranted(role, account, _msgSender());
            return true;
        } else {
            return false;
        }
    }
```

- contracts/src/DvPSettlement.sol#L233-L244

```
    function _grantRole(bytes32 role, address account) internal override returns (bool
granted) {
```

```
        granted = super._grantRole(role, account);
        if (granted && role == Roles.DEFAULT_ADMIN_ROLE) ++_defaultAdminCount;
    }

    function _revokeRole(bytes32 role, address account) internal override returns
(bool revoked) {
        if (role == Roles.DEFAULT_ADMIN_ROLE && _defaultAdminCount == 1 &&
hasRole(role, account)) {
            revert HTPErrors.LastDefaultAdminCannotBeRemoved(account);
        }
        revoked = super._revokeRole(role, account);
        if (revoked && role == Roles.DEFAULT_ADMIN_ROLE) --_defaultAdminCount;
    }
```

Solution

1.Before calling `super._grantRole()` in `UUPSRegistryBase._grantRole()`, reject a zero recipient address for `DEFAULT_ADMIN_ROLE`, reusing the existing `_requireRoleHolder()` helper.

2.Apply the same validation in `DvPSettlement._grantRole()` and add regression tests covering a zero-address admin grant followed by attempts to remove the last operational admin through either `renounceRole()` or `revokeRole()`.

Status

Fixed

[N4] [Information] Proxy Pattern Detection Cannot Fully Establish a Contract's Non-Proxy Status

Category: Design Logic Audit

Content

In the `RegistryImplementationValidation` library, `requireDirectCode()` first verifies that the target address has code, then uses `_containsProxyPattern()` to check whether its runtime bytecode matches a standard 45-byte EIP-1167 minimal proxy or contains the ERC-1967 implementation or beacon slot constants. This approach covers only a limited set of known proxy patterns and cannot conclusively determine whether the target is

a non-proxy contract. Proxies using custom storage slots, minimal proxies with appended data, and proxies that access a beacon through an immutable address without including these slot constants in their runtime bytecode may remain undetected. Passing this check therefore indicates only that none of the checked patterns was found; it does not establish the absence of proxy mechanisms or the immutability of execution logic.

- contracts/src/libraries/RegistryImplementationValidation.sol#L39-L66

```
    function _containsProxyPattern(address implementation, uint256 codeSize) private
view returns (bool) {
        bytes memory runtimeCode = new bytes(codeSize);
        assembly ("memory-safe") {
            extcodecopy(implementation, add(runtimeCode, 0x20), 0, codeSize)
        }
        if (codeSize == 45) {
            bytes32 prefixWord;
            bytes32 suffixWord;
            assembly ("memory-safe") {
                prefixWord := mload(add(runtimeCode, 0x20))
                suffixWord := mload(add(runtimeCode, 0x3e))
            }

            if ((prefixWord & EIP1167_PREFIX_MASK) == EIP1167_PREFIX && suffixWord ==
EIP1167_SUFFIX) {
                return true;
            }
        }
        if (codeSize < 32) return false;

        for (uint256 offset; offset <= codeSize - 32; ++offset) {
            bytes32 word;
            assembly ("memory-safe") {
                word := mload(add(add(runtimeCode, 0x20), offset))
            }
            if (word == ERC1967_IMPLEMENTATION_SLOT || word == ERC1967_BEACON_SLOT)
return true;
        }
```

```
        return false;
    }
```

Solution

It is recommended that privileged roles verify the target contract's source code, deployed bytecode, and proxy or upgrade mechanisms prior to registration; passing this function check should not be considered a substitute for manual review.

Status

Fixed

[N5] [Suggestion] Settlement Token Decimal Validation Is Inconsistent with the Range Supported by Market Rules

Category: Design Logic Audit

Content

In `SettlementTokenRegistry`, registering, activating, and resuming a settlement token configuration uses `TokenMetadataValidation.requireSettlementTokenDecimals()` to verify that the declared decimals match the token's `decimals()` return value, but does not restrict the value to the range supported by the protocol. This validation accepts values from `0` to `255`, whereas `MarketRuleRegistry` requires settlement token decimals not to exceed `PrincipalMath.MAX_ASSET_DECIMALS` (`36`) and requires `0 < priceDecimals <= settlementTokenDecimals`. Consequently, a settlement token with `0` decimals or more than `36` decimals can be registered and reach the `ACTIVE` state, but no market rule referencing it can pass validation. This delays detection of the incompatible configuration until market rule registration.

- contracts/src/registry/SettlementTokenRegistry.sol#L66, L226-L235

```
        TokenMetadataValidation.requireSettlementTokenDecimals(config.settlementToken,
config.settlementTokenDecimals);

    function _validateToken(SettlementTokenConfig memory config) private view {
```

```
        TokenMetadataValidation.requireTokenDeployment(
            config.settlementToken,
            config.tokenDeploymentKind,
            config.tokenCodeHash,
            config.implementation,
            config.implementationCodeHash
        );
        TokenMetadataValidation.requireSettlementTokenDecimals(config.settlementToken,
config.settlementTokenDecimals);
    }
```

- contracts/src/libraries/TokenMetadataValidation.sol#L14-L21, L31-L37

```
    function readDecimals(address token) internal view returns (bool valid, uint256
actual) {  // 读取代币代币精度
        (bool callSucceeded, bool returnWellFormed, bytes32 result) =
BoundedStaticCall.staticCallWordStatus(
            token, abi.encodeWithSelector(DECIMALS_SELECTOR), VALIDATION_GAS_LIMIT
        );
        actual = uint256(result);
        valid = callSucceeded && returnWellFormed && actual <= type(uint8).max;
        if (!valid) actual = INVALID_DECIMALS;
    }

    function requireSettlementTokenDecimals(address token, uint8 expected) internal
view {  // 检查结算代币的实际精度与声明的是否匹配
        (bool valid, uint256 actual) = readDecimals(token);

        if (!valid || actual != expected) {
            revert HTPErrors.SettlementTokenDecimalsMismatch(token, expected, actual);
        }
    }
```

- contracts/src/registry/MarketRuleRegistry.sol#L299-L308

```
        if (
            config.minCashPrincipal == 0 || config.minExecutableQuantity == 0 ||
config.quantityStep == 0
                || config.priceTick == 0 || config.priceDecimals == 0 ||
config.quantityDecimals == 0
                || config.quoteValidFrom == 0 || config.quoteValidFrom >
config.quoteValidUntil
                || config.quoteValidUntil > config.settlementValidUntil
                || asset.config.assetDecimals > PrincipalMath.MAX_ASSET_DECIMALS
                || settlement.config.settlementTokenDecimals >
PrincipalMath.MAX_ASSET_DECIMALS
                || config.quantityDecimals > asset.config.assetDecimals
                || config.priceDecimals > settlement.config.settlementTokenDecimals
        ) revert HTPErrors.InvalidMarketRule();
```

Solution

In the shared decimal validation used to register, activate, and resume settlement token configurations, require `1 <= settlementTokenDecimals <= PrincipalMath.MAX_ASSET_DECIMALS` while retaining the check against the on-chain `decimals()` return value. This aligns the accepted settlement token configuration range with the range currently supported by market rules.

Status

Fixed

[N6] [Suggestion] Inconsistent Gas Budgets Between Calculator Validation and Settlement

Category: Design Logic Audit

Content

In `PrincipalCalculatorRegistry`, `_validateCalculator()` probes `calculate()` with `200,000 gas` when registering, activating, or resuming a calculator configuration. However, `DvPCurrentStateValidator` provides only `100,000 gas` when calling the same interface during settlement. If a future calculator consumes more than `100,000 gas` but no more than `200,000 gas` for the probe input and satisfies the other validation checks, it

may pass registration, activation, and resumption validation, yet fail due to insufficient gas when settlement encounters similarly expensive inputs. This would prevent trades depending on that configuration from settling.

- contracts/src/registry/PrincipalCalculatorRegistry.sol#L242-L247

```
        (bool calculateCallSucceeded, bool calculateReturnWellFormed,,,,) =
BoundedStaticCall.staticCallFourWordsStatus(
            config.calculator,
            abi.encodeCall(IPrincipalCalculator.calculate,
(_probePriceData(config.assetClass), uint256(1), uint8(0))),
            RegistryImplementationValidation.VALIDATION_GAS_LIMIT
        );
        _requireCalculatorCallResult(config.calculatorConfigId,
calculateCallSucceeded, calculateReturnWellFormed);
```

- contracts/src/libraries/RegistryImplementationValidation.sol#L8

```
    uint256 internal constant VALIDATION_GAS_LIMIT = 200_000;
```

- contracts/src/validation/DvPCurrentStateValidator.sol#L43, L450-L469

```
    uint256 private constant CALCULATOR_GAS_LIMIT = 100_000;

        (
            bool calculatorCallSucceeded,
            bool calculatorReturnWellFormed,
            bytes32 calculatedPriceTermsHash,
            bytes32 quotedPriceWord,
            bytes32 unitCashPriceWord,
            bytes32 cashPrincipalWord
        ) = BoundedStaticCall.staticCallFourWordsStatus(
            context.calculator.config.calculator,
            abi.encodeWithSelector(
                IPrincipalCalculator.calculate.selector,
                params.snapshot.priceData,
```

```
                params.snapshot.executedQuantity,
                params.snapshot.assetDecimals
            ),
            CALCULATOR_GAS_LIMIT
        );
        if (!calculatorCallSucceeded || !calculatorReturnWellFormed) {
            return _externalUnknown();
        }
```

Solution

Align the gas budget for `calculate()` during calculator validation with the budget used during settlement, and use a shared constant to prevent the two settings from diverging.

Status

Fixed

[N7] [Information] Backdated Snapshot Time Allows a Limited Bypass of the Market Rule Quote Window

Category: Design Logic Audit

Content

In `MarketRuleRegistry`, `isMarketRuleServiceable()` uses `TradeTermsSnapshot.createdAt` to check whether a snapshot falls within the market rule's `[quoteValidFrom, quoteValidUntil]` window. This timestamp is bound through `snapshotHash` in the requester's signed `TradeIntent`, but the signature only proves approval of the value, not that the snapshot existed at the declared time. At settlement, the submitted snapshot and signatures alone cannot distinguish a historical snapshot created within the window from a new snapshot created after the window with a backdated timestamp.

After `quoteValidUntil`, while the rule's settlement window remains open and the rule is sealed, unpaused, and in `ACTIVE` or `LEGACY_SERVING` status, issuing a new quote and creating a new snapshot after the window requires cooperation between the maker and requester. The maker signs a new quote referencing that rule with an expiry that

remains valid; the requester creates a snapshot from the quote, backdates `createdAt` into the window, and signs a trade intent binding that snapshot. This allows the snapshot to pass these time checks. Both parties reference a market rule whose quote window has ended but whose settlement deadline has not passed; the quote and trade themselves must still be unexpired at settlement.

This path requires the maker's signed authorization of the referenced rule and quote terms. If the requester already holds a maker quote referencing that rule that remains valid at settlement, no further maker participation is needed to backdate a new snapshot based on that quote. The requester cannot independently replace the signed market rule ID, configuration hash, quantity bounds, or `stepQuantity`. The trade must still satisfy the quote's and market rule's quantity, price, and minimum principal requirements.

- contracts/src/registry/MarketRuleRegistry.sol#L233-L250

```
    function isMarketRuleServiceable(
        bytes32 configId,
        bytes32 expectedConfigHash,
        bytes32 assetVersionId,
        uint64 quoteCreatedAt,
        uint64 quoteExpiresAt,
        uint64 tradeDeadline
    ) external view returns (bool) {
        MarketRuleRecord storage record =
_getMarketRuleRegistryStorage().records[configId];
        MarketRuleConfig storage config = record.config;
        return record.exists && record.isSealed && !record.paused
            && (record.status == MarketRuleStatus.ACTIVE || record.status ==
MarketRuleStatus.LEGACY_SERVING)
            && config.configHash == expectedConfigHash &&
HTPHashing.hashMarketRuleConfig(config) == expectedConfigHash
            && config.assetVersionId == assetVersionId && quoteCreatedAt >=
config.quoteValidFrom
            && quoteCreatedAt <= config.quoteValidUntil && quoteCreatedAt <=
block.timestamp
            && tradeDeadline <= quoteExpiresAt && quoteExpiresAt <=
```

```
config.settlementValidUntil
            && tradeDeadline <= config.settlementValidUntil;
    }
```

- contracts/src/validation/DvPCurrentStateValidator.sol#L231-L237, L331-L341

```
        if (
            params.snapshot.createdAt == 0 || params.snapshot.createdAt >
block.timestamp
                || block.timestamp > params.snapshot.tradeDeadline
                || params.snapshot.tradeDeadline > params.snapshot.quoteExpiresAt
                || uint256(params.snapshot.tradeDeadline)
                    > uint256(params.snapshot.createdAt) +
ProtocolSecurityLimits.MAX_QUOTE_LIFETIME
        ) return _block(HTPErrors.InvalidSettlementTimeWindow.selector);

        MarketRuleReferences memory marketRuleReferences = MarketRuleReferences({
            configId: params.snapshot.marketRuleConfigId,
            configHash: params.snapshot.marketRuleConfigHash,
            assetVersionId: params.snapshot.assetVersionId,
            settlementToken: params.snapshot.settlementToken,
            settlementTokenConfigId: params.snapshot.settlementTokenConfigId,
            settlementTokenConfigHash: params.snapshot.settlementTokenConfigHash,
            createdAt: params.snapshot.createdAt,
            quoteExpiresAt: params.snapshot.quoteExpiresAt,
            tradeDeadline: params.snapshot.tradeDeadline
        });
```

- contracts/src/validation/DvPSignedPayloadValidator.sol#L94-L109

```
        bytes32 snapshotHash = HTPHashing.hashTradeTermsSnapshot(params.snapshot);

        if (params.tradeIntent.snapshotHash != snapshotHash) {
            return _hashFailure(HTPErrors.SnapshotHashMismatch.selector, snapshotHash,
params.tradeIntent.snapshotHash);
        }
```

```
        failure = _checkSignature(
            params.firmQuote.quotingMaker, quoteDigest, params.quotingMakerSignature,
reportExternalUncertainty
        );
        if (failure.reason != bytes4(0)) return failure;
        failure = _checkSignature(
            params.tradeIntent.requester,
            HTPHashing.hashTradeIntentDigest(domainSeparator, params.tradeIntent),
            params.requesterSignature,
            reportExternalUncertainty
        );
```

- contracts/src/libraries/ProtocolSecurityLimits.sol#L8

```
    uint64 internal constant MAX_QUOTE_LIFETIME = 1 hours;
```

Solution

Clarify whether `quoteValidUntil` is a strict cutoff for actual snapshot creation and whether the contract or a trusted service is responsible for enforcing it. If snapshots created after the window must be rejected, an authorized executor can verify creation time against trusted snapshot records. If on-chain verification that the snapshot existed before the cutoff is required, register its hash before the window closes or verify evidence from an independent trusted service that binds the snapshot hash and creation time. The evidence should match the submitted snapshot, associated maker quote, and market rule. Additional maker or requester signatures over a self-reported timestamp do not replace a trusted time reference.

These approaches either depend on the trustworthiness of an off-chain service or add on-chain operations and workflow costs. If the business accepts the existing bounded backdating tolerance, document that boundary and the verification responsibilities. Shortening the one-hour limit only reduces the exposure interval. Directly requiring `block.timestamp <= quoteValidUntil` would also reject historical trades created within the window and legitimately settled afterward, and should not be adopted without changing the business semantics.

Status

Acknowledged

[N8] [Suggestion] Closing a Market Rule Clears Its Pause Flag Without Emitting the Corresponding Event

Category: Design Logic Audit

Content

In `MarketRuleRegistry`, `closeMarketRule()` sets the rule's status to `CLOSED` and executes `record.paused = false`, but does not emit a `MarketRulePauseChanged` event. Consequently, when the rule being closed was paused, an off-chain indexer that reconstructs the pause flag independently from pause events may retain `paused = true`, diverging from the on-chain record. This issue affects event replay and index consistency; it does not make a closed rule serviceable again.

- contracts/src/registry/MarketRuleRegistry.sol#L203-L221

```
        MarketRuleStatus oldStatus = record.status;
        record.status = MarketRuleStatus.CLOSED;
        record.paused = false;
        if ($.currentConfigIds[record.config.assetVersionId] == configId) {
            $.currentConfigIds[record.config.assetVersionId] = bytes32(0);
            emit IMarketRuleRegistry.CurrentMarketRuleChanged(
                block.chainid, record.config.assetVersionId, configId, bytes32(0),
_msgSender()
            );
        }
        emit IMarketRuleRegistry.MarketRuleStatusChanged(
            block.chainid,
            configId,
            record.config.assetVersionId,
            oldStatus,
            MarketRuleStatus.CLOSED,
            evidenceHash,
            _msgSender()
```

```
        );
    }
```

Solution

When closing a rule changes its pause flag from `true` to `false`, also emit `MarketRulePauseChanged` with the governance action reason and the evidence hash used for closure. Alternatively, explicitly require off-chain indexers to clear the pause flag when processing a status change to `CLOSED`, so that event replay matches the on-chain record.

Status

Fixed

[N9] [Low] Settlement Token Adapter Failure Reports the Asset Adapter ID

Category: Design Logic Audit

Content

In `ConfigurationReferenceIntegrity`, `loadAdapters()` combines the serviceability of the asset adapter and the settlement token adapter into a single `serviceable` return value. When this value is `false`, `DvPCurrentStateValidator._loadAndValidateConfiguration()` returns only `AdapterInactive.selector`, without preserving the ID of the adapter that failed validation. The strict settlement path then calls `_revertFailure()`, which always populates `SettlementFailureContext.adapterConfigId` with `params.snapshot.adapterConfigId`. `StrictSettlementFailureMapper.revertFailure()` uses that field to construct the `AdapterInactive` error.

Consequently, when the asset and settlement token use different adapter configurations and only the settlement token adapter fails validation, the resulting error still contains the asset adapter ID, pointing to the serviceable asset adapter. This issue affects error accuracy and troubleshooting.

- contracts/src/validation/ConfigurationReferenceIntegrity.sol#L125-L135

```
    function loadAdapters(address dvp, AdapterReferences calldata expected)
        external
        view
        returns (AdapterRecord memory assetAdapter, AdapterRecord memory
settlementAdapter, bool serviceable)
    {
        assetAdapter = IDvPSettlement(dvp).adapterRegistry().getAdapter{gas:
REGISTRY_GAS_LIMIT}(expected.assetConfigId);
        settlementAdapter =
            IDvPSettlement(dvp).adapterRegistry().getAdapter{gas: REGISTRY_GAS_LIMIT}
(expected.settlementConfigId);
        serviceable = _adapterActive(assetAdapter, expected.assetConfigId,
expected.assetConfigHash)
            && _adapterActive(settlementAdapter, expected.settlementConfigId,
expected.settlementConfigHash);
    }
```

- contracts/src/validation/DvPCurrentStateValidator.sol#L149-L152, L301-L315, L543-L560

```
        (decision, reason) = _loadAndValidateConfiguration(params, context);
        if (decision != Decision.PASS) {
            _revertFailure(reason,
IDvPCurrentStateValidator.validateForSettlement.selector, params);
        }
```

```
        AdapterReferences memory adapterReferences = AdapterReferences({
            assetConfigId: params.snapshot.adapterConfigId,
            assetConfigHash: params.snapshot.adapterConfigHash,
            settlementConfigId: context.settlementToken.config.adapterConfigId,
            settlementConfigHash: context.settlementToken.config.adapterConfigHash
        });
        try REFERENCE_INTEGRITY.loadAdapters(DVP, adapterReferences) returns (
            AdapterRecord memory assetAdapter, AdapterRecord memory settlementAdapter,
bool serviceable
        ) {
            if (!serviceable) return _block(HTPErrors.AdapterInactive.selector);
```

```
            context.assetAdapter = assetAdapter;
            context.settlementAdapter = settlementAdapter;
        } catch {
            return _externalUnknown();
        }
```

```
    function _revertFailure(bytes4 reason, bytes4 externalSelector, SettlementParams
calldata params) private view {
        STRICT_FAILURE_MAPPER.revertFailure(
            reason,
            externalSelector,
            SettlementFailureContext({
                chainId: params.snapshot.chainId,
                dvpContract: params.snapshot.dvpContract,
                feePolicyEpoch: params.snapshot.feePolicyEpoch,
                assetVersionId: params.snapshot.assetVersionId,
                settlementTokenConfigId: params.snapshot.settlementTokenConfigId,
                adapterConfigId: params.snapshot.adapterConfigId,
                principalCalculatorConfigId:
params.snapshot.principalCalculatorConfigId,
                feeVersionId: params.snapshot.feeVersionId,
                marketRuleConfigId: params.snapshot.marketRuleConfigId
            })
        );
        revert HTPErrors.InvalidDvpValidator(address(STRICT_FAILURE_MAPPER));
    }
```

- contracts/src/validation/StrictSettlementFailureMapper.sol#L60-L62

```
        if (reason == HTPErrors.AdapterInactive.selector) {
            revert HTPErrors.AdapterInactive(context.adapterConfigId);
        }
```

Solution

Preserve the configuration ID of the adapter that fails serviceability validation and propagate it through the call chain

to the error mapper, so that `AdapterInactive` contains the ID of the failing adapter. Avoid always using the asset adapter ID when the failing side has not been identified.

Status

Fixed

[N10] [Medium] Missing Maker Signature Binding Allows Unilateral Trade ID Replacement by the Requester

Category: Design Logic Audit

Content

The maker's signature covers only `FirmQuote`, which does not contain `tradeIdHash`. For this field, the contract requires a nonzero value, equality between `TradeIntent` and `TradeTermsSnapshot`, and the requester's signature, but does not verify its binding to the maker's quote and the original trade. The requester can therefore replace both occurrences of `tradeIdHash`, recompute and update `TradeIntent.snapshotHash`, and sign a new `TradeIntent` while retaining the original maker quote and signature. Settlement still requires submission by an authorized executor and satisfaction of the other settlement conditions.

This may cause the off-chain trade record associated with a `tradeIdHash` to differ from the trade contents actually executed on-chain. Additionally, the requester can replace their trade's `tradeIdHash` with another party's unused ID and occupy it by front-running the original trade's settlement, causing the original trade to revert with `TradeAlreadyExecuted` when attempting to settle under that ID.

- contracts/src/libraries/SettlementParamsValidation.sol#L84

```
        if (params.tradeIntent.tradeIdHash != params.snapshot.tradeIdHash) return
bytes32("tradeIdHash");
```

- contracts/src/validation/DvPSignedPayloadValidator.sol#L94-L109

```
        bytes32 snapshotHash = HTPHashing.hashTradeTermsSnapshot(params.snapshot);

        if (params.tradeIntent.snapshotHash != snapshotHash) {
            return _hashFailure(HTPErrors.SnapshotHashMismatch.selector, snapshotHash,
params.tradeIntent.snapshotHash);
        }

        failure = _checkSignature(
            params.firmQuote.quotingMaker, quoteDigest, params.quotingMakerSignature,
reportExternalUncertainty
        );
        if (failure.reason != bytes4(0)) return failure;
        failure = _checkSignature(
            params.tradeIntent.requester,
            HTPHashing.hashTradeIntentDigest(domainSeparator, params.tradeIntent),
            params.requesterSignature,
            reportExternalUncertainty
        );
```

- contracts/src/DvPSettlement.sol#L461, L488, L771-L776

```
        if (usedTradeIdHash[tradeIdHash]) return
(HTPErrors.TradeAlreadyExecuted.selector, tradeIdHash);
```

```
        usedTradeIdHash[tradeIdHash] = true;
```

```
        bytes32 tradeIdHash = params.tradeIntent.tradeIdHash;
        bytes32 snapshotHash = params.tradeIntent.snapshotHash;
        bytes32 assetVersionId = params.snapshot.assetVersionId;
        bytes32 eventTopic = SETTLEMENT_EXECUTED_TOPIC;
        assembly ("memory-safe") {
            log4(data, 0x400, eventTopic, tradeIdHash, snapshotHash, assetVersionId)
```

Solution

Establish and verify a binding between `tradeIdHash` and the trade contents authorized by both parties, for example by including it in the payload signed by the maker, so that the requester cannot replace the ID while retaining the original maker signature.

Status

Fixed

[N11] [Low] Market Rule Violations Are Misreported as External Call Failures During Strict Settlement

Category: Design Logic Audit

Content

In the `DvPCurrentStateValidator` contract, `_validatePrincipalAndFees()` returns `MarketRulePriceTickViolation` when the quoted price does not align with the price tick, or `MarketRuleMinimumPrincipalViolation` when the cash principal is below the market rule minimum. `preCheck()` preserves these business rejection reasons, but the strict settlement path forwards the reason together with `IPrincipalCalculator.calculate.selector` to the error mapper. Because `StrictSettlementFailureMapper.revertFailure()` does not map either error, execution reaches the `ExternalCallFailed` fallback and misreports a deterministic market rule violation as a calculator call failure.

The invalid transaction is still rejected before transfers, so this issue does not bypass market rules or directly cause fund loss. However, the inconsistent failure reasons returned by `preCheck()` and `settle()` for the same input may mislead monitoring, alerting, and operational troubleshooting by presenting a business parameter issue as a calculator failure.

- contracts/src/validation/DvPCurrentStateValidator.sol#L164-L172, L475-L480

```
        (decision, reason) = _validatePrincipalAndFees(params, context);
        if (decision != Decision.PASS) {
            if (reason == HTPErrors.FeeConstraintHashMismatch.selector) {
                revert HTPErrors.FeeConstraintHashMismatch(
```

```
                    params.snapshot.feeConstraintHash,
HTPHashing.hashFeeConstraint(context.feeVersion.config)
                );
            }
            _revertFailure(reason, IPrincipalCalculator.calculate.selector, params);
        }
```

```
        if (uint256(quotedPriceWord) % context.marketRulePriceTick != 0) {
            return _block(HTPErrors.MarketRulePriceTickViolation.selector);
        }
        if (uint256(cashPrincipalWord) < context.marketRuleMinCashPrincipal) {
            return _block(HTPErrors.MarketRuleMinimumPrincipalViolation.selector);
        }
```

- contracts/src/validation/StrictSettlementFailureMapper.sol#L69-L82

```
        if (reason == HTPErrors.MarketRuleSnapshotMismatch.selector) {
            revert HTPErrors.MarketRuleSnapshotMismatch(context.marketRuleConfigId);
        }
        if (reason == HTPErrors.MarketRuleQuantityViolation.selector) {
            revert HTPErrors.MarketRuleQuantityViolation(context.marketRuleConfigId);
        }
        if (reason == HTPErrors.FeeResolverNoMatch.selector) {
            revert HTPErrors.FeeResolverNoMatch(context.feePolicyEpoch,
context.assetVersionId);
        }
        if (reason == HTPErrors.CashPrincipalZero.selector) revert
HTPErrors.CashPrincipalZero();

        revert HTPErrors.ExternalCallFailed(externalSelector == bytes4(0)
                ? IDvPCurrentStateValidator.validateForSettlement.selector
                : externalSelector);
```

Solution

Add strict-path error handling for `MarketRulePriceTickViolation` and

`MarketRuleMinimumPrincipalViolation`, passing the market rule configuration ID, actual quoted price or cash principal, and the corresponding threshold required to construct each error, so that `settle()` preserves the accurate business rejection reason.

Status

Fixed

[N12] [Suggestion] Participant Type Validation Order Makes the Dedicated Error Branch Unreachable

Category: Others

Content

In the `StrictSettlementFailureMapper` contract, `_participantValidation()` first calls `_participantActive()` to check whether the requester record is active and its type matches `requesterType`, and only then checks whether `requesterType` is `INVESTOR` or `MAKER`. The current `ParticipantType` enum contains only `NONE`, `INVESTOR`, and `MAKER`, while `ParticipantRegistry` rejects registration with type `NONE`. Under normal registry state, `requesterType == NONE` therefore returns `ParticipantInactive` due to the type mismatch before reaching the dedicated `ParticipantRelationshipInvalid` branch, making that branch unreachable.

- contracts/src/validation/StrictSettlementFailureMapper.sol#L206-L211, L277-L284

```
        if (!_participantActive(requesterRecord, requester, requesterType)) {
            return (Decision.BLOCK, HTPErrors.ParticipantInactive.selector,
requester);
        }
        if (requesterType != ParticipantType.INVESTOR && requesterType !=
ParticipantType.MAKER) {
            return (Decision.BLOCK, HTPErrors.ParticipantRelationshipInvalid.selector,
address(0));
        }
```

```
    function _participantActive(ParticipantRecord memory record, address wallet,
ParticipantType expectedType)
        private
        view
        returns (bool)
    {
        return record.wallet == wallet && record.version != 0 &&
record.participantType == expectedType
            && record.status == ParticipantStatus.ACTIVE && record.validUntil >
block.timestamp;
    }
```

- contracts/src/registry/ParticipantRegistry.sol#L153-L155

```
        if (input.participantType == ParticipantType.NONE) {
            revert HTPErrors.InvalidParticipantType(input.participantType);
        }
```

Solution

Move the `requesterType` allowlist check before participant registry queries so that an invalid type directly returns `ParticipantRelationshipInvalid`, rather than being classified as `ParticipantInactive` by the earlier participant status and type matching checks.

Status

Fixed

[N13] [Suggestion] Missing Nonzero Evidence Hash Validation When Creating or Updating Participants

Category: Others

Content

In the `ParticipantRegistry` contract, `_validateParticipantInput()`, which is called by `setParticipant()`, checks only the wallet address, participant identity hash, participant type, and validity period,

without requiring a nonzero `evidenceHash`. An account with `CONFIGURER_ROLE` can therefore create a participant with a zero evidence hash, or overwrite an existing nonzero evidence hash with zero when updating the validity period, provided the other parameter and version checks pass. This is inconsistent with the nonzero evidence requirements for pausing, resuming, and revoking participants in the same contract, as well as for `EligibilityRegistry.setEligibility()`.

- contracts/src/registry/ParticipantRegistry.sol#L48-L56, L66-L68, L149-L159

```
            $.participants[input.wallet] = ParticipantRecord({
                wallet: input.wallet,
                participantIdHash: input.participantIdHash,
                participantType: input.participantType,
                status: ParticipantStatus.ACTIVE,
                validUntil: input.validUntil,
                version: 1,
                evidenceHash: input.evidenceHash
            });
```

```
            current.validUntil = input.validUntil;
            current.version = actualVersion + 1;
            current.evidenceHash = input.evidenceHash;
```

```
    function _validateParticipantInput(ParticipantInput calldata input) private view {
        if (input.wallet == address(0) || input.participantIdHash == bytes32(0)) {
            revert HTPErrors.InvalidParticipantInput();
        }
        if (input.participantType == ParticipantType.NONE) {
            revert HTPErrors.InvalidParticipantType(input.participantType);
        }
        if (input.validUntil <= block.timestamp) {
            revert HTPErrors.InvalidParticipantValidity(input.validUntil,
uint64(block.timestamp));
```

```
        }
    }
```

Solution

Require `input.evidenceHash != bytes32(0)` in `_validateParticipantInput()` and revert with `EvidenceHashRequired` when it is zero, aligning participant creation and updates with the evidence requirements of other lifecycle operations.

Status

Fixed

[N14] [Information] ERC-1967 Slot Constant Scanning May Reject Legitimate Direct Contracts

Category: Others

Content

In the `RegistryImplementationValidation` library, `_containsProxyPattern()` scans the target contract's entire runtime bytecode one byte at a time. Any consecutive 32 bytes matching the ERC-1967 implementation or beacon slot constant cause the contract to be classified as having a proxy pattern. The check does not distinguish whether the constant is used for proxy delegation, returned by an ordinary getter, or stored in an unreachable data region. Consequently, legitimate direct contracts that do not delegate calls as a proxy but retain these constants in their runtime bytecode may also be misclassified.

`requireDirectCode()` then reverts with `UpgradeableImplementationForbidden`, preventing the affected Adapter or Calculator from being registered. `TokenMetadataValidation.requireTokenDeployment()` instead reverts with `TokenDeploymentKindMismatch` for a Token declared as `DIRECT`. Even with otherwise valid configuration and code hashes, affected contracts cannot pass admission checks and may require modification and redeployment. A third-party Token that the protocol cannot modify may therefore be incompatible with registration as `DIRECT`.

- contracts/src/libraries/RegistryImplementationValidation.sol#L26-L28, L58-L64

```
        if (_containsProxyPattern(implementation, codeSize)) {
            revert HTPErrors.UpgradeableImplementationForbidden(implementation);
        }
```

```
        for (uint256 offset; offset <= codeSize - 32; ++offset) {
            bytes32 word;
            assembly ("memory-safe") {
                word := mload(add(add(runtimeCode, 0x20), offset))
            }
            if (word == ERC1967_IMPLEMENTATION_SLOT || word == ERC1967_BEACON_SLOT)
return true;
        }
```

- contracts/src/libraries/TokenMetadataValidation.sol#L61-L63

```
        if (kind == TokenDeploymentKind.DIRECT &&
RegistryImplementationValidation.hasProxyShape(token)) {
            revert HTPErrors.TokenDeploymentKindMismatch(token, kind);
        }
```

Solution

Document that this is a conservative proxy pattern check and that non-proxy contracts containing the relevant slot constants may also be rejected. Review the source code, deployed bytecode, and actual call behavior before onboarding to establish compatibility.

Status

Acknowledged

[N15] [Medium] Risk of Excessive Privilege

Category: Authority Control Vulnerability Audit

Content

1.In `DvPSettlement`, `DEFAULT_ADMIN_ROLE` can grant and revoke all roles, but the last default admin cannot be

removed. `PAUSER_ROLE` can globally pause settlement. `GOVERNANCE_ROLE` can resume settlement or permanently retire the contract after it has been paused. `EXECUTOR_ROLE` can submit settlements, which must still pass signature, state, and replay-protection checks. Pausing, resuming, and retirement all require a reason and an evidence hash.

- contracts/src/DvPSettlement.sol#L233, L238, L361, L374, L388, L403

```
function _grantRole(bytes32 role, address account) internal override returns (bool
granted) { ... }

function _revokeRole(bytes32 role, address account) internal override returns (bool
revoked) { ... }

function pauseGlobal(ReasonCode reason, bytes32 evidenceHash) external override
onlyRole(Roles.PAUSER_ROLE) nonReentrant { ... }

function unpauseGlobal(ReasonCode reason, bytes32 evidenceHash) external override
onlyRole(Roles.GOVERNANCE_ROLE) nonReentrant { ... }

function retire(ReasonCode reason, bytes32 evidenceHash) external override
onlyRole(Roles.GOVERNANCE_ROLE) nonReentrant { ... }

function settle(SettlementParams calldata params) external override {
    _checkRole(Roles.EXECUTOR_ROLE, _msgSender());
    ...
}
```

2.All registries inherit `UUPSRegistryBase`, directly or indirectly, and share the following role structure:

- `DEFAULT_ADMIN_ROLE`: Grants and revokes all roles, but cannot remove the last default admin.
- `CONFIGURER_ROLE`: Registers configurations or creates and updates business records.
- `GOVERNANCE_ROLE`: Performs governance actions such as activation, resumption, deprecation, and revocation; registers dependent DvP contracts and exempts retired dependent DvP contracts from upgrade

pause checks.

- `PAUSER_ROLE`: Pauses configurations or business records.
- `UPGRADER_ROLE`: Performs contract upgrades. At least one dependent DvP must be registered, and all dependent DvP contracts without a retirement exemption must be globally paused.

Role assignments are independent across registries. Default admins do not automatically hold business roles, but can grant those roles to themselves.

- contracts/src/registry/UUPSRegistryBase.sol#L34, L55, L61, L71, L87, L112

```
function _initializeRegistryBase( address defaultAdmin, address governance, address upgrader, address pauser, address configurer ) internal onlyInitializing { ... }

function _grantRole(bytes32 role, address account) internal override returns (bool granted) { ... }

function _revokeRole(bytes32 role, address account) internal override returns (bool revoked) { ... }

function _authorizeUpgrade(address) internal virtual override onlyRole(Roles.UPGRADER_ROLE) { ... }

function registerDependentDvp(address dvp, bytes32 evidenceHash) external onlyRole(Roles.GOVERNANCE_ROLE) { ... }

function exemptRetiredDependentDvp(address dvp, bytes32 evidenceHash) external onlyRole(Roles.GOVERNANCE_ROLE) { ... }
```

3.The registry-specific business permissions are as follows:

- `AdapterRegistry`, `PrincipalCalculatorRegistry`, `SettlementTokenRegistry`, and `AssetVersionRegistry`: Configurers register configurations; governance activates, resumes, and deprecates them.

- `ParticipantRegistry`: Configurers create participants directly in the active state and update existing records; governance resumes and permanently revokes participants.
- `EligibilityRegistry`: Configurers create and update eligibility records and decisions without separate governance activation; governance resumes and permanently revokes eligibility records.
- `MarketRuleRegistry`: Configurers register draft rules; governance seals, activates, resumes, and closes them.
- `FeeRegistry`: Configurers register fee versions; governance activates, resumes, and deprecates fee versions, and creates, seals, activates, moves to legacy serving, and closes fee policy epochs.

- contracts/src/registry/AdapterRegistry.sol#L47, L62, L91, L99, L122

```
function registerAdapter(AdapterConfig calldata config) external
onlyRole(Roles.CONFIGURER_ROLE) { ... }

function activateAdapter(bytes32 adapterConfigId, bytes32 expectedConfigHash) external
onlyRole(Roles.GOVERNANCE_ROLE) { ... }

function pauseAdapter(bytes32 adapterConfigId, ReasonCode reason, bytes32
evidenceHash) external onlyRole(Roles.PAUSER_ROLE) { ... }

function unpauseAdapter( bytes32 adapterConfigId, bytes32 expectedConfigHash,
ReasonCode reason, bytes32 evidenceHash ) external onlyRole(Roles.GOVERNANCE_ROLE) {
... }

function deprecateAdapter(bytes32 adapterConfigId, ReasonCode reason, bytes32
evidenceHash) external onlyRole(Roles.GOVERNANCE_ROLE) { ... }
```

- contracts/src/registry/PrincipalCalculatorRegistry.sol#L47, L80, L105, L113, L136

```
function registerPrincipalCalculator(PrincipalCalculatorConfig calldata config)
external onlyRole(Roles.CONFIGURER_ROLE) { ... }

function activatePrincipalCalculator(bytes32 calculatorConfigId, bytes32
```

```
expectedConfigHash) external onlyRole(Roles.GOVERNANCE_ROLE) { ... }

function pausePrincipalCalculator(bytes32 calculatorConfigId, ReasonCode reason,
bytes32 evidenceHash) external onlyRole(Roles.PAUSER_ROLE) { ... }

function unpausePrincipalCalculator( bytes32 calculatorConfigId, bytes32
expectedConfigHash, ReasonCode reason, bytes32 evidenceHash ) external
onlyRole(Roles.GOVERNANCE_ROLE) { ... }

function deprecatePrincipalCalculator(bytes32 calculatorConfigId, ReasonCode reason,
bytes32 evidenceHash) external onlyRole(Roles.GOVERNANCE_ROLE) { ... }
```

- contracts/src/registry/SettlementTokenRegistry.sol#L54, L84, L108, L137, L164

```
function registerSettlementToken(SettlementTokenConfig calldata config) external
onlyRole(Roles.CONFIGURER_ROLE) { ... }

function activateSettlementToken(bytes32 configId, bytes32 expectedConfigHash)
external onlyRole(Roles.GOVERNANCE_ROLE) { ... }

function pauseSettlementToken(bytes32 configId, ReasonCode reason, bytes32
evidenceHash) external onlyRole(Roles.PAUSER_ROLE) { ... }

function unpauseSettlementToken( bytes32 configId, bytes32 expectedConfigHash,
ReasonCode reason, bytes32 evidenceHash ) external onlyRole(Roles.GOVERNANCE_ROLE) {
... }

function deprecateSettlementToken(bytes32 configId, ReasonCode reason, bytes32
evidenceHash) external onlyRole(Roles.GOVERNANCE_ROLE) { ... }
```

- contracts/src/registry/AssetVersionRegistry.sol#L111, L147, L171, L208, L236

```
function registerAssetVersion(AssetVersionConfig calldata config) external
onlyRole(Roles.CONFIGURER_ROLE) { ... }

function activateAssetVersion(bytes32 assetVersionId, bytes32 expectedConfigHash)
```

```
external onlyRole(Roles.GOVERNANCE_ROLE) { ... }

function pauseAssetVersion(bytes32 assetVersionId, ReasonCode reason, bytes32
evidenceHash) external onlyRole(Roles.PAUSER_ROLE) { ... }

function unpauseAssetVersion( bytes32 assetVersionId, bytes32 expectedConfigHash,
ReasonCode reason, bytes32 evidenceHash ) external onlyRole(Roles.GOVERNANCE_ROLE) {
... }

function deprecateAssetVersion(bytes32 assetVersionId, ReasonCode reason, bytes32
evidenceHash) external onlyRole(Roles.GOVERNANCE_ROLE) { ... }
```

- contracts/src/registry/ParticipantRegistry.sol#L35, L86, L105, L124

```
function setParticipant(ParticipantInput calldata input, uint64 expectedVersion)
external onlyRole(Roles.CONFIGURER_ROLE) { ... }

function pauseParticipant(address wallet, uint64 expectedVersion, ReasonCode reason,
bytes32 evidenceHash) external onlyRole(Roles.PAUSER_ROLE) { ... }

function unpauseParticipant(address wallet, uint64 expectedVersion, ReasonCode reason,
bytes32 evidenceHash) external onlyRole(Roles.GOVERNANCE_ROLE) { ... }

function revokeParticipant(address wallet, uint64 expectedVersion, bytes32
evidenceHash) external onlyRole(Roles.GOVERNANCE_ROLE) { ... }
```

- contracts/src/registry/EligibilityRegistry.sol#L29, L67, L81, L96

```
function setEligibility(EligibilityInput calldata input, uint64 expectedRecordVersion)
external onlyRole(Roles.CONFIGURER_ROLE) { ... }

function pauseEligibility(bytes32 eligibilityKey, uint64 expectedVersion, ReasonCode
reason, bytes32 evidenceHash) external onlyRole(Roles.PAUSER_ROLE) { ... }

function unpauseEligibility(bytes32 eligibilityKey, uint64 expectedVersion, ReasonCode
reason, bytes32 evidenceHash) external onlyRole(Roles.GOVERNANCE_ROLE) { ... }
```

```
function revokeEligibility(bytes32 eligibilityKey, uint64 expectedVersion, bytes32
evidenceHash) external onlyRole(Roles.GOVERNANCE_ROLE) { ... }
```

- contracts/src/registry/MarketRuleRegistry.sol#L63, L102, L114, L166, L179, L194

```
function registerMarketRule(MarketRuleConfig calldata config) external
onlyRole(Roles.CONFIGURER_ROLE) { ... }

function sealMarketRule(bytes32 configId, bytes32 expectedConfigHash) external
onlyRole(Roles.GOVERNANCE_ROLE) { ... }

function activateMarketRule(bytes32 configId, bytes32 expectedConfigHash) external
onlyRole(Roles.GOVERNANCE_ROLE) { ... }

function pauseMarketRule(bytes32 configId, ReasonCode reason, bytes32 evidenceHash)
external onlyRole(Roles.PAUSER_ROLE) { ... }

function unpauseMarketRule(bytes32 configId, bytes32 expectedConfigHash, ReasonCode
reason, bytes32 evidenceHash) external onlyRole(Roles.GOVERNANCE_ROLE) { ... }

function closeMarketRule(bytes32 configId, bytes32 evidenceHash) external
onlyRole(Roles.GOVERNANCE_ROLE) { ... }
```

- contracts/src/registry/FeeRegistry.sol#L137, L174, L187, L221, L232, L239, L271, L302, L319, L345

```
function createFeePolicyEpoch(FeePolicyEpochConfig calldata config) external
onlyRole(Roles.GOVERNANCE_ROLE) { ... }

function sealFeePolicyEpoch(uint64 feePolicyEpoch, bytes32 expectedConfigHash)
external onlyRole(Roles.GOVERNANCE_ROLE) { ... }

function activateFeePolicyEpoch(uint64 feePolicyEpoch, bytes32 expectedConfigHash)
external onlyRole(Roles.GOVERNANCE_ROLE) { ... }

function moveFeePolicyEpochToLegacyServing(uint64 feePolicyEpoch, bytes32
```

```
evidenceHash) external onlyRole(Roles.GOVERNANCE_ROLE) { ... }

function closeFeePolicyEpoch(uint64 feePolicyEpoch, bytes32 evidenceHash) external
onlyRole(Roles.GOVERNANCE_ROLE) { ... }

function registerFeeVersion(FeeConfig calldata config) external
onlyRole(Roles.CONFIGURER_ROLE) { ... }

function activateFeeVersion(bytes32 feeVersionId, bytes32 expectedConfigHash) external
onlyRole(Roles.GOVERNANCE_ROLE) { ... }

function pauseFeeVersion(bytes32 feeVersionId, ReasonCode reason, bytes32
evidenceHash) external onlyRole(Roles.PAUSER_ROLE) { ... }

function unpauseFeeVersion( bytes32 feeVersionId, bytes32 expectedConfigHash,
ReasonCode reason, bytes32 evidenceHash ) external onlyRole(Roles.GOVERNANCE_ROLE) {
... }

function deprecateFeeVersion(bytes32 feeVersionId, ReasonCode reason, bytes32
evidenceHash) external onlyRole(Roles.GOVERNANCE_ROLE) { ... }
```

Solution

In the short term, transferring owner ownership to multisig contracts is an effective solution to avoid single-point risk. But in the long run, it is a more reasonable solution to implement a privilege separation strategy and set up multiple privileged roles to manage each privileged function separately. And the authority involving user funds should be managed by the community, and the EOA address can manage the authority involving emergency contract suspension. This ensures both a quick response to threats and the safety of user funds.

Status

Acknowledged

5 Audit Result

Audit Number	Audit Team	Audit Date	Audit Result
0X002610230001	SlowMist Security Team	2026.09.22 - 2026.10.23	Medium Risk

Summary conclusion: The SlowMist security team audited the project through manual review and the team's analysis tool. During the audit, we identified 3 medium-risk findings, 3 low-risk findings, 6 suggestions, and 3 informational findings.

6 Statement

SlowMist issues this report with reference to the facts that have occurred or existed before the issuance of this report, and only assumes corresponding responsibility based on these.

For the facts that occurred or existed after the issuance, SlowMist is not able to judge the security status of this project, and is not responsible for them. The security audit analysis and other contents of this report are based on the documents and materials provided to SlowMist by the information provider till the date of the insurance report (referred to as "provided information"). SlowMist assumes: The information provided is not missing, tampered with, deleted or concealed. If the information provided is missing, tampered with, deleted, concealed, or inconsistent with the actual situation, the SlowMist shall not be liable for any loss or adverse effect resulting therefrom. SlowMist only conducts the agreed security audit on the security situation of the project and issues this report. SlowMist is not responsible for the background and other conditions of the project.



Official Website

www.slowmist.com



E-mail

team@slowmist.com



Twitter

@SlowMist_Team



Github

https://github.com/slowmist